

Cloud Computing Architecture

Semester project report

Group 24

Orofino Antonino - 22-956-908

Paleari Lorenzo - 22-943-047

Sainz Martinez Julián - 22-953-467

Systems Group
Department of Computer Science
ETH Zurich
May 24, 2023

Part 3 [34 points]

1. [17 points] With your scheduling policy, run the entire workflow **3 separate times**. For each run, measure the execution time of each PARSEC job, as well as the latency outputs of memcached running with a steady client load of 30K QPS. For each PARSEC application, compute the mean and standard deviation of the execution time¹ across three runs. Also compute the mean and standard deviation of the total time to complete all jobs. Fill in the table below. Finally, compute the SLO violation ratio for memcached for the three runs; the number of data points with 95th percentile latency > 1ms, as a fraction of the total number of data points while the jobs are running.

Answer:

job name	mean time [s]	std [s]
blackscholes	68.66	2.49
canneal	126	6.48
dedup	22.66	0.94
ferret	139.33	2.05
freqmine	124.00	0.82
radix	86.00	2.16
vips	69.66	1.70
total time	143.66	1.25

- Generally, the memcached p95 latency is around 0.5ms. The SLO violation ratio is 0.0 for all runs, since the number of data points with 95th percentile latency > 1ms is 0 for all runs.

Create 3 bar plots (one for each run) of memcached p95 latency (y-axis) over time (x-axis) with annotations showing when each PARSEC job started and ended, also indicating the machine they are running on. Using the augmented version of mcperf, you get two additional columns in the output: `ts_start` and `ts_end`. Use them to determine the width of each bar while the height should represent the p95 latency. Align the x axis so that $x = 0$ coincides with the starting time of the first container. Use the colors proposed in this template (you can find them in `main.tex`). For example, use the **vips** color to annotate when vips started.

Plots:

¹You should only consider the runtime, excluding time spans during which the container is paused.

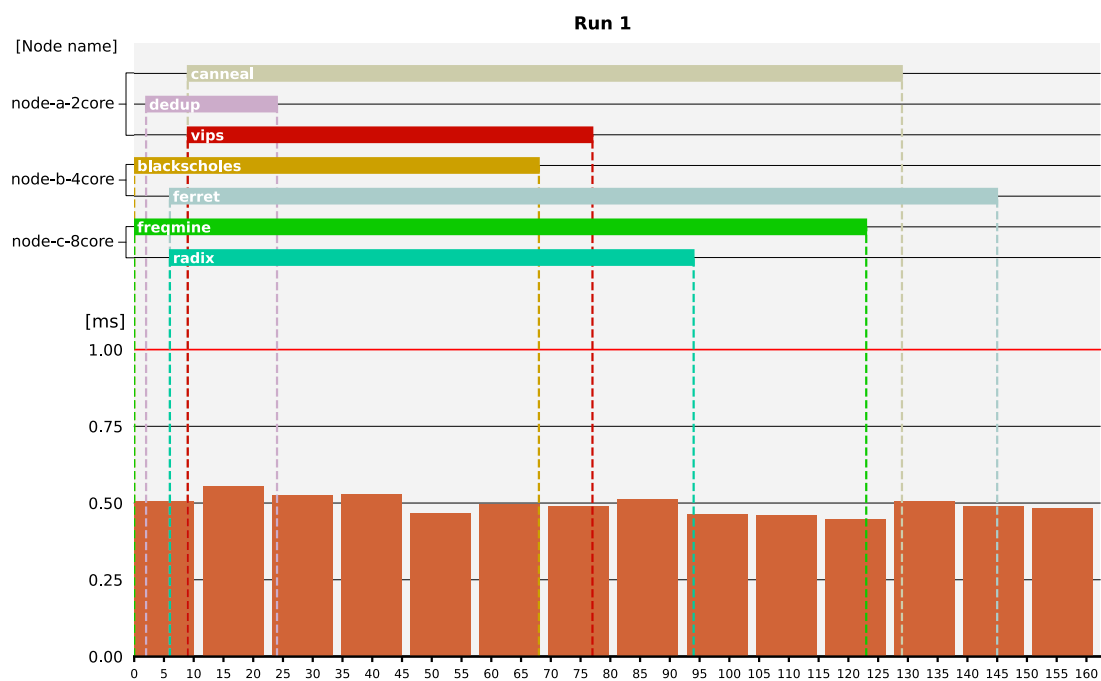


Figure 1: Run 1

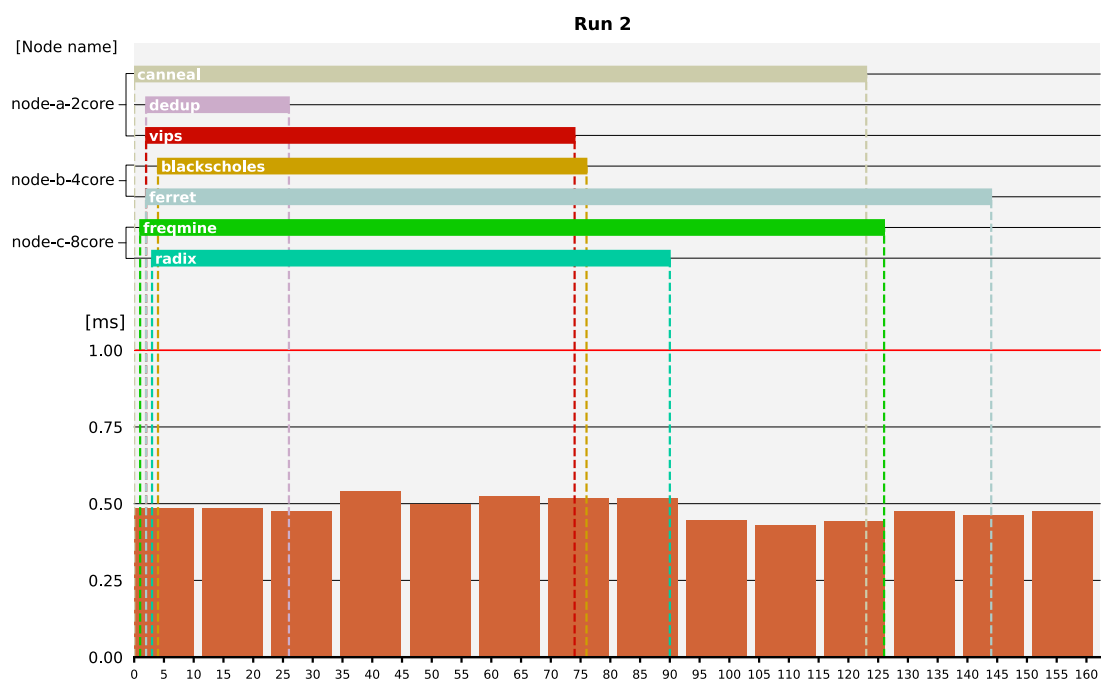


Figure 2: Run 2

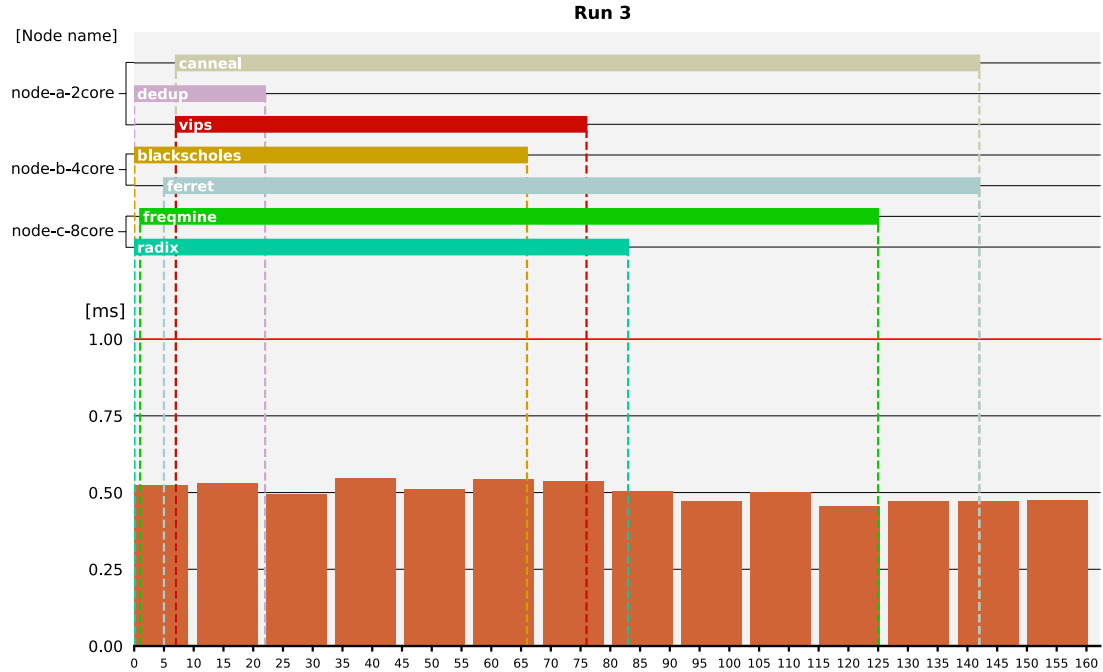


Figure 3: Run 3

2. [17 points] Describe and justify the “optimal” scheduling policy you have designed.

- Which node does memcached run on? Why?

Answer: Memcached is running on node-c-8core with its CPU affinity set to CPU 0. We took into account several factors while deciding the node for memcached.

Firstly, memcached does not require much CPU and memory while operating at a constant 30K QPS. Testing it, we found out that it needs just 0.5 CPU and 32MB memory to run smoothly.

Secondly, after consulting the documentation, we discovered that the e2-standard-8 machine, despite having the highest number of cores, has weaker CPUs compared to the other machines. Therefore, we decided to reserve the faster machines for Parsec jobs.

We requested at least 0.5 CPU and 32 MB memory for memcached and limited to maximum 0.75 CPU and 64 MB respectively using Kubernetes. This configuration prevented other Parsec jobs running on the same machine and same core from using all the CPUs and memory which would have resulted in a higher latency for memcached.

- Which node does each of the 7 PARSEC jobs run on? Why?

Answer:

- **blackscholes**: Node-b, because it performs good with four threads and node-b has four cores, providing a good environment for it to run efficiently. Assigning it less core while ferret is running on the same node would have resulted in a higher execution time of blackscholes and as a consequence a higher ferret execution time.
- **canneal**: Node-a, as it does not demand many resources but needs two cores to minimize its execution time.

- **dedup**: Node-a, despite being a very demanding job, it is quick to run and one core on Node-a can accommodate it.
- **ferret**: Node-b, as it's high in CPU demand and scales well with the available cores. It is one of the most demanding jobs CPU and memory wise, we assigned also blackscholes in this node because it is not a very demanding job and they fit together.
- **fraqmine**: Node-c, as it scales well with the number of cores and the eight cores of node-c helps minimize its execution time. It is the longest job to run, so we assigned it to the node with the most cores.
- **radix**: Node-c, as it has the least resource requirements, and just one of the low performing cores in node-c is sufficient for it. Being very low demanding do not interfere much with fraqmine neither interfere at all with memcached.
- **vips**: Node-a, because it requires high memory transfer speed which node-a provides being the best spec machine amongst all.

Further explanation on our motivations is given in the section regarding design choices at the end of Part 3.

- Which jobs run concurrently / are colocated? Why?

Answer: On node-a, canneal, dedup, and vips are run concurrently because it minimizes the total execution time compared to running them sequentially.

On node-b, blackscholes and ferret are run concurrently. Even though ferret's execution time increases by 20% when colocated with blackscholes, the total time for running both is less than running them separately.

On node-c, fraqmine and radix are run concurrently. Radix shares the core with the memcached server because it has low resource needs and doesn't interfere with the server's performance.

- In which order did you run 7 PARSEC jobs? Why?

Order: blackscholes, canneal, dedup, ferret, fraqmine, radix, vips

Why: All the jobs are started **concurrently** on the three nodes, hence there is no order in which they are started.

- How many threads have you used for each of the 7 PARSEC jobs? Why?

Answer:

- **blackscholes**: Four threads because it performs better with four threads and 4 cores. Assigning it less threads would have increased its execution time exceeding the achieved runtime. Using two cores has been tested but it is not as efficient as using four cores because it takes long to complete and interfere longer with ferret that is a demanding job running on the same node.
- **canneal**: Two threads as it is run on node-a which has two cores, and this minimizes its execution time. Using more thread is not beneficial as it doesn't scale well with more than two thread.
- **dedup**: One thread, as it runs on one core of node-a and completes quickly. Moreover it doesn't scale well with more threads. Just one core was assigned also to interfere less with canneal that is running on the same node.
- **ferret**: Four threads, as it is run on node-b, which has four cores, and it scales well with more cores.

- **fregmine**: Eight threads because it is run on node-c which has eight cores and it scales well with the number of cores. Using four threads would have increased its execution time significantly.
- **radix**: One thread because it has low resource requirements and shares a core with the memcached server on node-c. It is actually run with just 0.1 CPU and a hard limit on 0.25 to not interfere with the memcached server. It does not interfere also with fregmine that is a demanding job running on the same node.
- **vips**: One thread because it is quick to run and it can share the core only with canneal, using two cores would mean share the core with three process concurrently. It is assigned in the other core respect to dedup, so they do not interfere with each other ends quickly and leave the resources to canneal.
- Which files did you modify or add and in what way? Which Kubernetes features did you use?

Answer: We revised seven YAML configuration files corresponding to the seven PAR-SEC jobs. The modifications involved setting the number of threads each job should use, defining the CPU affinity via the `taskset` command and the `nodeSelector` property to specify on which node each job should run.

Additionally, the YAML configuration file for the memcached server was adjusted to specify the resource request using Kubernetes feature **Resources** using both **Request** and **limit** to set how much CPU and memory a job is assured to use and the maximum it can use respectively.

A Python script was developed to automate the process of job creation, memcached server setup, and the implementation of the scheduling mechanism. Find below the output of the help command of this script which describes the different functionalities it provides.

```
python3 main.py --help
usage: main [-h] {create,status,start,stop,destroy,ssh,test,time} ...
```

Command line interface for cluster operations

```
positional arguments:
{create,status,start,stop,destroy,ssh,test,time}
  create                create cluster
  status                get status
  start                 start service
  stop                  stop service
  destroy               destroy cluster
  ssh                   ssh to a node
  test                  test a scheduler
  time                  analyze results
```

```
optional arguments:
-h, --help            show this help message and exit
```

- Describe the design choices, ideas and trade-offs you took into account while creating your scheduler (if not already mentioned above):

Answer: The main design consideration for the scheduler was to minimize total execution time of all jobs, while not interfering much with the memcached execution. This was achieved by:

- Co-locating jobs that are compatible in terms of resource usage. For instance, ferret and blackscholes are run together on node-b, while canneal, dedup, and vips are colocate on node-a.
- Matching jobs with the right nodes based on their resource demands and the node’s capabilities. For example, vips is put on node-a because it requires high memory transfer speed.
- Running jobs concurrently on each node instead of sequentially to minimize total execution time.
- Adjusting the number of threads each job uses according to the number of cores in each node and the individual job’s performance under various thread counts. For instance, blackscholes runs with four threads on node-b because it performs better under these conditions, while freqmine uses all eight cores of node-c because it scales well with the number of cores.

The major trade-off in this scheduler design is slowing down the individual execution time of each job to minimize the total execution time. For example, the execution time of ferret increases by about 20% when run together with blackscholes on node-b, but the total execution time of both jobs is less when run together compared to running them consecutively.

Another trade-off is the choice to limit the number of cores/threads some jobs can use despite their ability to scale well with more cores. This decision is made to ensure other processes or jobs on the same node are not negatively affected. For example, radix, despite scaling the best with the number of cores, is limited to use only 0.25 core on node-c to avoid interfering with memcached, it is still very fast to complete and it would be a waste of CPU power to assign to it more CPUs. Similarly, vips, despite scaling well, is limited to use only one core on node-a to not compete with three jobs concurrently, also it is faster than canneal to complete and therefore canneal has been assigned two cores to complete faster.

The design of the scheduler involves a delicate balance of maximizing the utilization of available resources, managing co-locations to minimize total execution time, and respecting the specific needs and characteristics of each job. These decisions have been taken looking at the characteristics of each job and the resources available on each node, and by running experiments to determine the best number of threads for each job. We took into account also the scalability and resource demand of each jobs tested on part 2.

Looking at the documentation we realize that the most powerful CPU are in node-a, following node-b and node-c so we run every job alone in every machine with various number of threads and core to find how much time each jobs takes on each configuration to be able later with all the information to decide better where and how schedule the jobs. The results of this experiment can be found in Table 1

Table 1: Runtime (s) of PARSEC jobs on various configurations

Blackscholes			
Number of cores	Runtime in node-a	Runtime in node-b	Runtime in node-c
1	01:04	01:33	02:09
2	00:37	01:02	01:16
4		00:49	00:49
8			00:38
Canneal			
Number of cores	Runtime in node-a	Runtime in node-b	Runtime in node-c
1	02:32	03:16	05:06
2	01:32	02:05	03:10
4		01:32	02:05
8			01:40
Dedup			
Number of cores	Runtime in node-a	Runtime in node-b	Runtime in node-c
1	00:19	00:25	00:39
2	00:17	00:24	00:19
4		00:24	00:15
8			00:15
Vips			
Number of cores	Runtime in node-a	Runtime in node-b	Runtime in node-c
1	01:09	01:35	01:42
2	00:50	01:05	00:56
4		00:59	00:31
8			00:25
Freqmine			
Number of cores	Runtime in node-a	Runtime in node-b	Runtime in node-c
1	04:11	05:14	08:15
2	02:08	02:38	03:40
4		1:50	01:55
8			01:21
Ferret			
Number of cores	Runtime in node-a	Runtime in node-b	Runtime in node-c
1	05:22	06:46	08:15
2	02:44	03:27	04:08
4		02:27	02:06
8			01:46
Radix			
Number of cores	Runtime in node-a	Runtime in node-b	Runtime in node-c
1	00:31	00:40	00:59
2	00:17	00:25	00:31
4		00:16	00:16
8			00:16

Part 4 [76 points]

1. [20 points] Use the following `mcperf` command to vary QPS from 5K to 125K in order to answer the following questions:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
  --noload -T 16 -C 4 -D 4 -Q 1000 -c 4 -t 5 \
  --scan 5000:125000:5000
```

a) [10 points] How does memcached performance vary with the number of threads (T) and number of cores (C) allocated to the job? In a single graph, plot the 95th percentile latency (y-axis) vs. QPS (x-axis) of memcached (running alone, with no other jobs collocated on the server) for the following configurations (one line each):

- Memcached with $T=1$ thread, $C=1$ core
- Memcached with $T=1$ thread, $C=2$ cores
- Memcached with $T=2$ threads, $C=1$ core
- Memcached with $T=2$ threads, $C=2$ cores

Label the axes in your plot. State how many runs you averaged across (we recommend three runs) and include error bars. The readability of your plot will be part of your grade.

Plots:

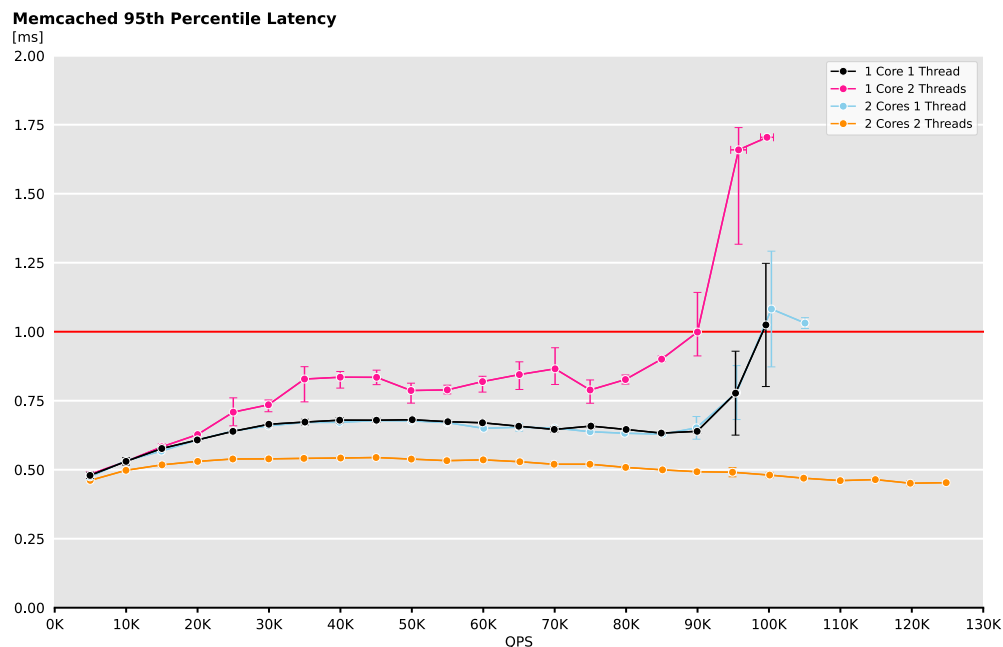


Figure 4: 95th percentile latency vs. QPS for memcached averaged over 3 runs

Error bars are present in all plots, but some measurements were too consistent and the error is too little to be visible on some part of the graph. 105K QPS in 2 core 1 thread is reached just one time out of 3 runs, we reported it for completeness.

What do you conclude from the results in your plot? Summarize in 2-3 brief sentences how memcached performance varies with the number of threads and cores.

Summary: The results indicate that memcached performance is least efficient with 1 core and 2 threads, likely due to the overhead of managing an extra thread on a single core and due to the threads competing with each other to use the core. Maximum performance is observed with a configuration of 2 cores and 2 threads, which allows the threads to operate in parallel. Other configurations, specifically 2 cores with 1 thread, yield similar performance to a 1 core 1 thread setup, due to the singular thread being confined to one core.

b) [2 points] To support the highest load in the trace (125K QPS) without violating the 1ms latency SLO, how many memcached threads (T) and CPU cores (C) will you need?

Answer: From the plot it is clearly visible that the only configuration that did not violate the SLO at 125K QPS was 2 Cores 2 Threads. Therefore, we will need 2 Cores and 2 Threads to support the highest load in the trace without violating the 1ms latency SLO.

c) [1 point] Assume you can change the number of cores allocated to memcached dynamically as the QPS varies from 5K to 125K, but the number of threads is fixed when you launch the memcached job. How many memcached threads (T) do you propose to use to guarantee the 1ms 95th percentile latency SLO while the load varies between 5K to 125K QPS?

Answer: If you can change the number of cores allocated to memcached dynamically as the QPS varies from 5K to 125K, but the number of threads is fixed when you launch the memcached job, then you should use 2 memcached threads (T) to guarantee the 1ms 95th percentile latency SLO while the load varies between 5K to 125K QPS. This is because the configuration of 2 cores and 2 threads provided the best performance in the plot and is the only one capable of handling 125K QPS, by fixing the number of threads to 2, you can ensure that the threads can be scheduled on different cores as the number of cores allocated to memcached changes dynamically. While it is true that 2 cores and 1 thread is the configuration with the worst performance, in case the load becomes too high we can change to 2 cores and 2 threads to ensure that the SLO is not violated.

d) [7 points] Run memcached with the number of threads T that you proposed in (c) and measure performance with $C = 1$ and $C = 2$. Use the aforementioned `mcperf` command to sweep QPS from 5K to 125K.

Measure the CPU utilization on the memcached server at each 5-second load time step.

Plot the performance of memcached using 1-core ($C = 1$) and using 2 cores ($C = 2$) in **two separate graphs**, for $C = 1$ and $C = 2$, respectively. In each graph, plot QPS on the x-axis, ranging from 0 to 130K. In each graph, use two y-axes. Plot the 95th percentile latency on the left y-axis. Draw a dotted horizontal line at the 1ms latency SLO. Plot the CPU utilization (ranging from 0% to 100% for $C = 1$ or 200% for $C = 2$) on the right y-axis. For simplicity, we do not require error bars for these plots.

Plots:

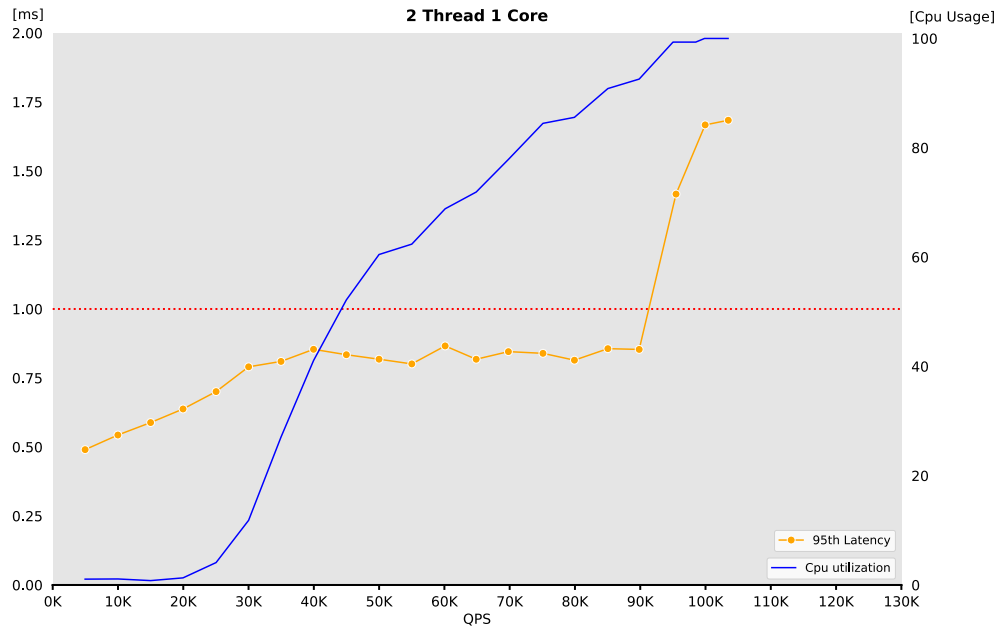


Figure 5: 95th percentile latency vs. QPS for memcached with 1 core

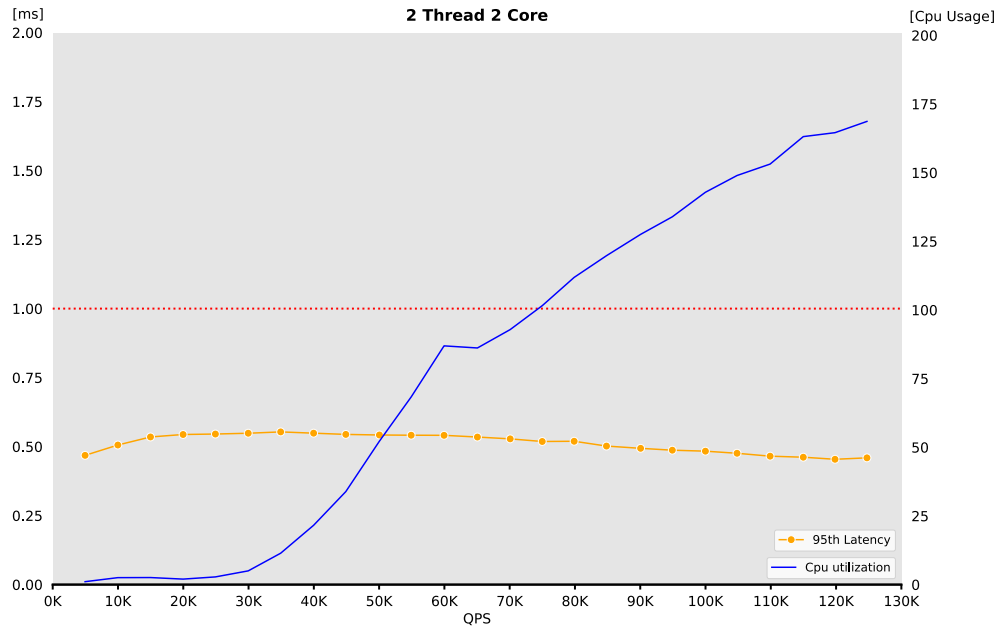


Figure 6: 95th percentile latency vs. QPS for memcached with 2 cores

2. [17 points] You are now given a dynamic load trace for memcached, which varies QPS randomly between 5K and 100K in 10 second time intervals. Use the following command to run this trace:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
    --noload -T 16 -C 4 -D 4 -Q 1000 -c 4 -t 1800 \
    --qps_interval 10 --qps_min 5000 --qps_max 100000
```

Note that you can also specify a random seed in this command using the `--qps_seed` flag.

For this and the next questions, feel free to reduce the mcperf measurement duration (`-t` parameter, now fixed to 30 minutes) as long as you have at the end at least 1 minute of memcached running alone.

Design and implement a controller to schedule memcached and the PARSEC benchmarks on the 4-core VM. The goal of your scheduling policy is to successfully complete all PARSEC jobs as soon as possible without violating the 1ms 95th percentile latency for memcached. **Your controller should not assume prior knowledge of the dynamic load trace. You should design your policy to work well regardless of the random seed.** The PARSEC jobs need to use the native dataset, i.e., provide the option `-i native` when running them. Also make sure to check that all the PARSEC jobs complete successfully and do not crash. Note that PARSEC jobs may fail if given insufficient resources.

Describe how you designed and implemented your scheduling policy. Include the source code of your controller in the zip file you submit.

- Brief overview of the scheduling policy (max 10 lines):

Answer: The scheduling policy manages CPU resources between memcached and the PARSEC benchmark jobs using Docker containers. Two job queues handle single-core and dual-core jobs. The policy keeps a check on CPU usage and adjusts memcached's CPU affinity as required. Memcached primarily operates on core 0, occasionally utilizing core 1 based on load. High CPU usage triggers a pause on the core 1 job, allocating its CPU to memcached. Low usage lets the paused job resume or starts a new one, confining memcached to core 0. High CPU usage is defined as above 80% on a single core or less than 130% combined on two cores. The policy assigns core 1 to canneal, vips, blackscholes, and dedup, while cores 2 and 3 are allocated to ferret, freqmine, and radix. After radix, ferret and freqmine complete, other jobs can also utilize cores 2 and 3.

- How do you decide how many cores to dynamically assign to memcached? Why?

Answer: Memcached starts running on only one core. The policy assigns two cores to it when the CPU utilization is over 80%. When the CPU is less than 130% when summing both the utilization of core 0 and 1, memcached is scaled back down to core 0.

We decided to use this value as a threshold after seeing the results from the combined measure of latency and CPU using 2 threads of the previous point. We observed that until 80% of CPU utilization, the latency was not affected. However, after that point, the latency started to increase and we need to switch.

Additionally the 130% worked well, because we noticed that when splitting the load between 2 processors the load on each processor did not exactly halved. Rather, it was higher than expected. Thus the combined load was set to a combined 130% because

after fusing the load of the two processors into one, the load was typically around 70% and not maxed at 100% as would be expected from putting together a combined load of 130%. This behavior may be due to the fact that the operating system's scheduling mechanism may lead to uneven distribution of load, creating an imbalance in CPU usage. Imperfectly balanced threads and additional overheads from multithreading could consume more CPU time.

This strategy is designed with the aim of optimizing resource use while ensuring smooth operations and effective system performance. When the system is under high load (CPU utilization above 80%), assigning an extra core to memcached helps to improve its performance by allowing it to process more data concurrently assuring the SLO of 1ms 95th percentile latency is not violated, as high CPU utilization correlates to the number of QPS that memcached is receiving.

Conversely, when the CPU utilization is relatively low (combined utilization of both cores less than 130%), it is more efficient to limit memcached to a single core, freeing up resources for other processes or jobs that might require them, reducing the overall time of execution of the PARSEC jobs. This dynamic and responsive approach ensures a balanced, efficient use of system resources.

- How do you decide how many cores to assign each PARSEC job? Why?

Answer:

- **blackscholes**: This job is assigned to core 1 sequentially following the scheduling policy, given that although it can benefit from parallelism its execution time is not one of the longest.
- **canneal**: Similar to blackscholes, this job is also assigned to core 1 in sequence. According to the analysis in Part 1-2 Canneal is not a CPU-intensive task and it is also not one of the longest jobs. Therefore, it is not necessary to assign more than one core to it.
- **dedup**: Dedup, too, is allocated to core 1 sequentially. This job is not very CPU-intensive, and also does not benefit from parallelism as much as other jobs. Moreover it is a very fast job.
- **ferret**: Ferret is assigned to cores 2 and 3 exclusively, according to the policy. As a parallel search engine, ferret is a CPU-intensive task that benefits from multiple cores to improve its parallel execution.
- **freqmine**: Similar to ferret, freqmine also utilizes cores 2 and 3. As an application which performs frequent itemset mining, freqmine is a computationally intensive task and thus requires multiple cores to optimize its performance. Ferret together with freqmine are the two benchmarks that take the longer to execute, therefore any improvement on these will have a big impact on the overall runtime. Therefore, two cores were assigned to both of them. As mentioned before, in case ferret and freqmine finished before the other jobs running in core 1, these jobs will be also allowed to use these two core, as this was a scenario that happened sometimes in out testing.
- **radix**: Radix is assigned to cores 2 and 3, before ferret and freqmine started. Radix was observed to have a good response to parallelism and to be really fast. Additionally, ferret and freqmine run in around 10 minutes total. Thus, there was some time available once they had finished to run one of the other jobs. These times matched more closely to the execution time of radix.

- **vips**: Like blackscholes, canneal, and dedup, vips is allocated to core 1 sequentially. As an image processing system, vips requires high computational resources, but according to the analysis in Part 1-2 is less intensive than ferret or freqmine and, therefore, is allocated only one core.

- How many threads do you use for each of the PARSEC job? Why?

Answer:

- **blackscholes**: For blackscholes, we allocate only one thread as the job runs on a single core (core 1). Multiple threads wouldn't improve performance significantly and might increase overhead due to context switching.
- **canneal**: As stated, one thread is provided to canneal as it is allocated a single core. More threads would increase overhead without noticeable performance gain.
- **dedup**: Dedup also receives one thread, due to its allocation on a single core. Similar reasoning to canneal and blackscholes applies here.
- **ferret**: Ferret, being allocated cores 2 and 3, uses two threads to fully exploit the parallel processing capacity of the two cores. This approach significantly enhances its parallel execution performance.
- **freqmine**: Freqmine also uses two threads corresponding to its allocation on cores 2 and 3. This allows the job to leverage the full computational potential of the two cores, improving its performance.
- **radix**: Radix follows a similar policy to freqmine, and ferret, with two threads assigned due to its allocation on two cores (core 2 and 3).
- **vips**: Vips is also allocated a single core (core 1) and thus uses one thread. Similar to blackscholes, canneal, and dedup, multiple threads would not improve performance significantly and might increase overhead due to context switching.

- Which jobs run concurrently / are collocated and on which cores? Why?

Answer: No job is run concurrently with another on the same core. Instead, jobs are allocated to cores in a sequential manner to avoid resource contention and ensure optimal execution. Memcached starts on core 0 and can expand to core 1 if the CPU utilization exceeds 80%. On core 1, jobs blackscholes, canneal, dedup, and vips run sequentially, not concurrently. These jobs are queued up for execution on core 1 in the order they're mentioned. They may, however, be temporarily paused if memcached needs to expand to core 1 due to high CPU utilization.

Jobs ferret, freqmine, and radix are allocated cores 2 and 3 exclusively, running sequentially on these cores. Once ferret and freqmine are finished, the other jobs initially set to run only on core 1 are allowed to run on cores 2 and 3 as well.

While some jobs can be putted together to improve the total runtime of the scheduler like we did in part 3 the overhead to pause and unpaue two or more jobs on core 1 can become too much to respond quickly to the changing QPS of memcached. ferret and freqmine which run on cores 2 and 3 are not run with other jobs because this is only going to slow down the total runtime.

This design is in place to avoid resource contention and ensure each job gets exclusive access to a core for a period of time. Memcached, as a critical service, gets preferential treatment, with the ability to claim an additional core if needed.

- In which order did you run the PARSEC jobs? Why?

Order:

- Core 1: `canneal`, `vips`, `blackscholes`, `dedup`
- Core 2 and 3: `radix`, `ferret`, `freqmine`

Why: There is actually not a reason for this ordering as all jobs are run sequentially and, therefore, there is no contention between them.

- How does your policy differ from the policy in Part 3? Why?

Answer: In Part 3, the policy did not involve dynamically adjusting the number of running jobs based on the CPU utilization of the system and neither changing the number of cores of memcached or other applications.

In contrast, the policy in Part 4 involves dynamically adjusting the number of running containers based on the CPU utilization of the system. The policy uses the `psutil` library to monitor the CPU utilization of the system, and pauses and unpauses containers based on the CPU utilization. If the CPU utilization is too high, some of the containers are paused to free up resources. If the CPU utilization is low, some of the paused containers are resumed to make use of the available resources.

The main reason for this difference is that the policy in Part 4 is more flexible and can adapt to changes in the CPU utilization of the system. By dynamically adjusting the number of running containers, the policy can ensure that the system is using the available resources efficiently, while also maximizing the performance of the system. In contrast, the policy in Part 3 is more static and does not adapt to changes in the CPU utilization of the system.

We also changed the policy from running all in parallel to running just two jobs in parallel in different cores due to the reduced number of total core available.

- How did you implement your policy? e.g., docker cpu-set updates, taskset updates for memcached, pausing/unpausing containers, etc.

Answer: The policy implementation involves adjusting the CPU affinity of the 'memcached' process using the `taskset` command, creating and running Docker containers for PARSEC jobs with specified CPU cores, pausing and unpausing Docker containers, and updating the cores of Docker containers. The implementation also involves monitoring the CPU utilization and adjusting the CPU affinity of 'memcached' accordingly. We created several functions to implement the policy:

- The `adjust_memcached_resources` function uses the `subprocess` module to call `taskset` with the following parameters:
 - * `cp` followed by the CPU cores to which 'memcached' should be bound, specified as a comma-separated list of integers.
 - * `memcached_pid`, which is the process ID of the 'memcached' process. This is used to reschedule memcached to core 0 or to core 0 and 1 depending on the load.
- The `create_job` function uses the Docker API to create a Docker container with the following parameters:
 - * `image`: the name of the Docker image to use for the container.
 - * `command`: the command to run inside the container.
 - * `cpuset_cpus`: a string specifying the CPU cores to which the container should be bound, in the format "0,1" for example.
 - * `detach=True`: a boolean indicating whether the container should be detached from the calling terminal.

- * **name**: a name for the container.

We use this function to create all of the containers at the beginning so that we can just simply start them when they are needed.

- The **update_job_cores** function uses the Docker API to update the CPU cores of a Docker container with the following parameters:

- * **container**: the name or ID of the container to update.

- * **cpuset_cpus**: a string specifying the new CPU cores to which the container should be bound.

This is used in our scheduler if, once the tasks initially assigned to cores 2 and 3 have finished, there are tasks still pending that were assigned to core 1.

- The **pause_job** and **unpause_job** functions respectively pause and resume a Docker container using the Docker API. The **pause_job** function takes the following parameters:

- * **container**: the name or ID of the container to pause.

The **unpause_job** function takes the same parameter. These functions are used to pause and resume containers based on the CPU utilization. If the CPU utilization is too high, and memcached is scheduled to core 1 as well we pause the running container on this core to free up resources. If the CPU utilization is low, we resume some of the paused containers to make use of the available resources.

The ‘main’ function creates and runs Docker containers for PARSEC jobs with specified CPU cores, monitors the CPU utilization, and adjusts the CPU affinity of ‘memcached’ accordingly. It also pauses and unpauses Docker containers and updates the cores of Docker containers based on the CPU utilization using the functions described above.

- Describe the design choices, ideas and trade-offs you took into account while creating your scheduler (if not already mentioned above):

Answer: Most of these aspects were already explained before. Perhaps the trade-offs aspect requires more detail. The main trade-off that was made was to decide which tasks to run exclusively in cores 2 and 3. This was done according to the results of Part 1-2 as ferret and freqmine were the more CPU intensive. Additionally, the choice to run radix also in these cores, was made due to the fact that its execution time matched precisely the additional time that jobs allocated to core 1 took to complete once ferret and freqmine were done.

3. [23 points] Run the following mcperf memcached dynamic load trace:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
--noload -T 16 -C 4 -D 4 -Q 1000 -c 4 -t 1800 \
--qps_interval 10 --qps_min 5000 --qps_max 100000 \
--qps_seed 3274
```

Measure memcached and PARSEC performance when using your scheduling policy to launch workloads and dynamically adjust container resource allocations. Run this workflow 3 separate times. For each run, measure the execution time of each PARSEC job, as well as the latency outputs of memcached. For each PARSEC application, compute the mean and standard deviation of the execution time across three runs. Compute the mean and standard

deviation of the total time to complete all jobs. Fill in the table below. Also, compute the SLO violation ratio for memcached for each of the three runs; the number of data points with 95th percentile latency $> 1\text{ms}$, as a fraction of the total number of datapoints. You should only report the runtime, excluding time spans during which the container is paused.

Answer:

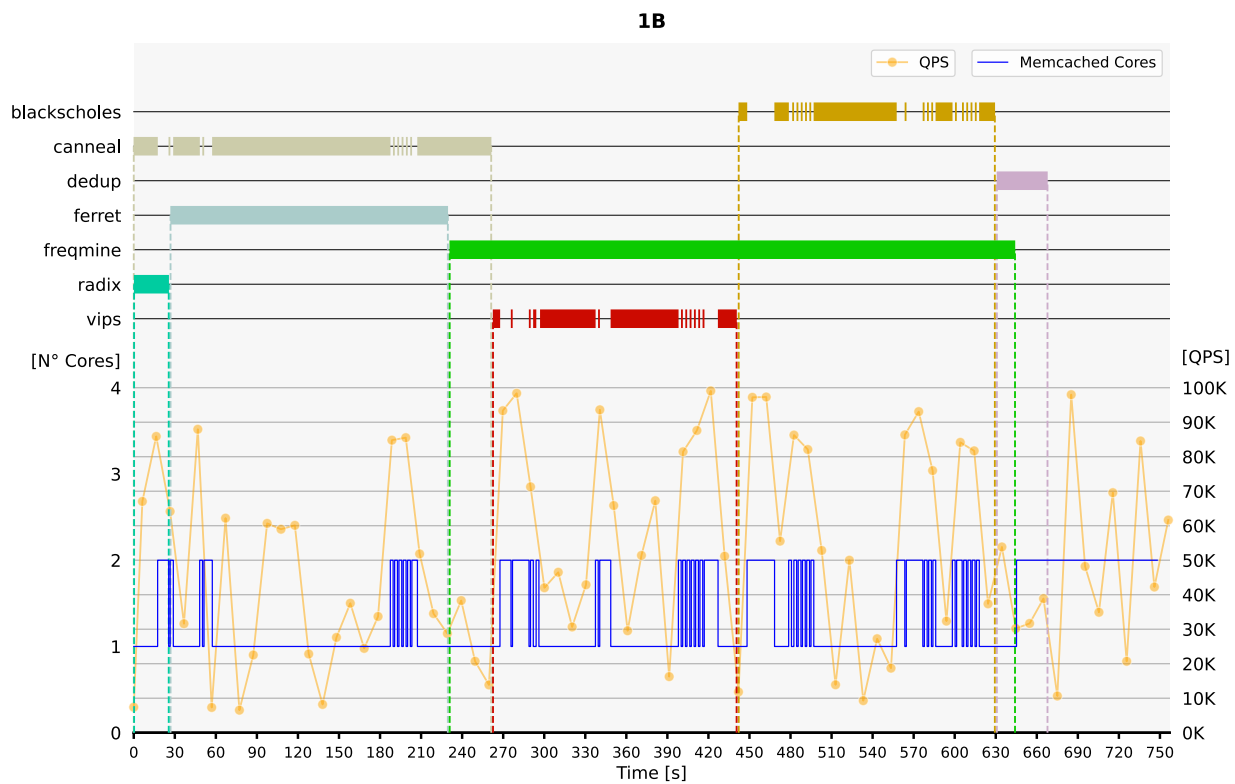
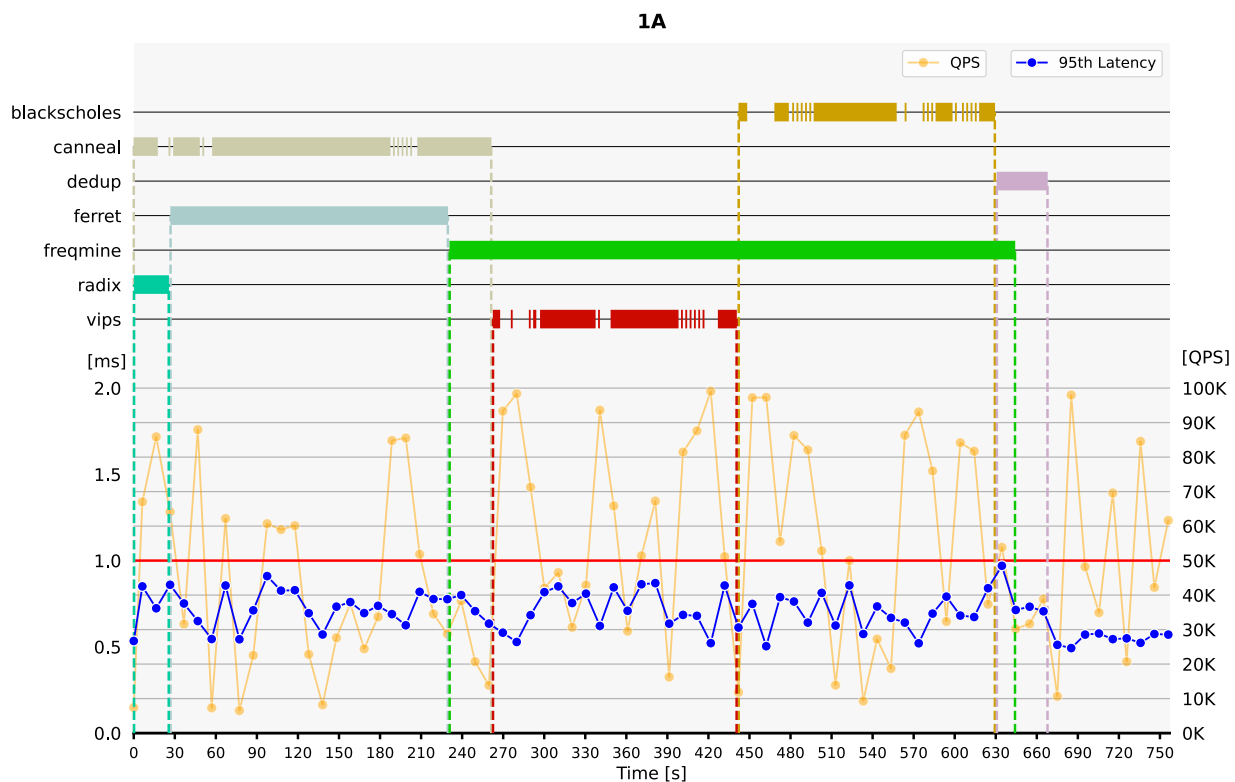
job name	mean time [s]	std [s]
blackscholes	114.85	4.60
canneal	237.43	20.45
dedup	38.01	0.76
ferret	213.68	13.85
frequine	411.50	3.95
radix	26.38	1.39
vips	118.06	2.62
total time	680.33	32.42

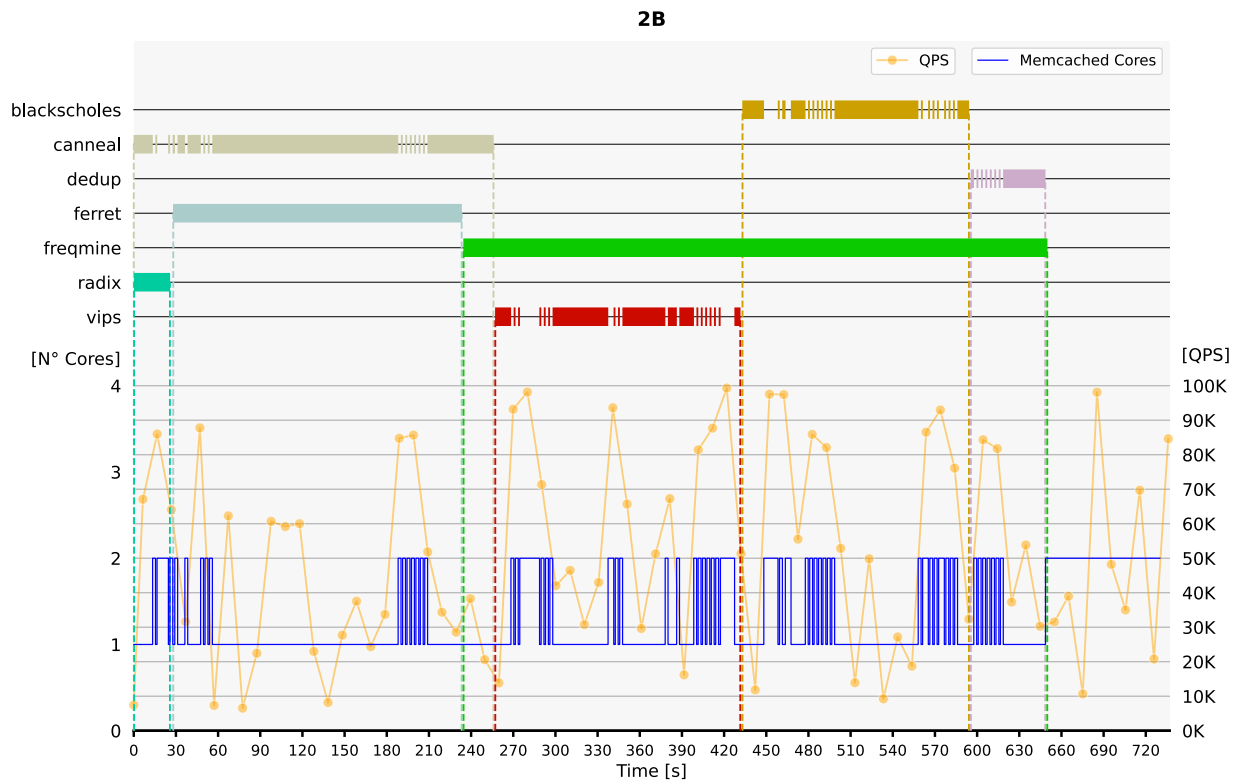
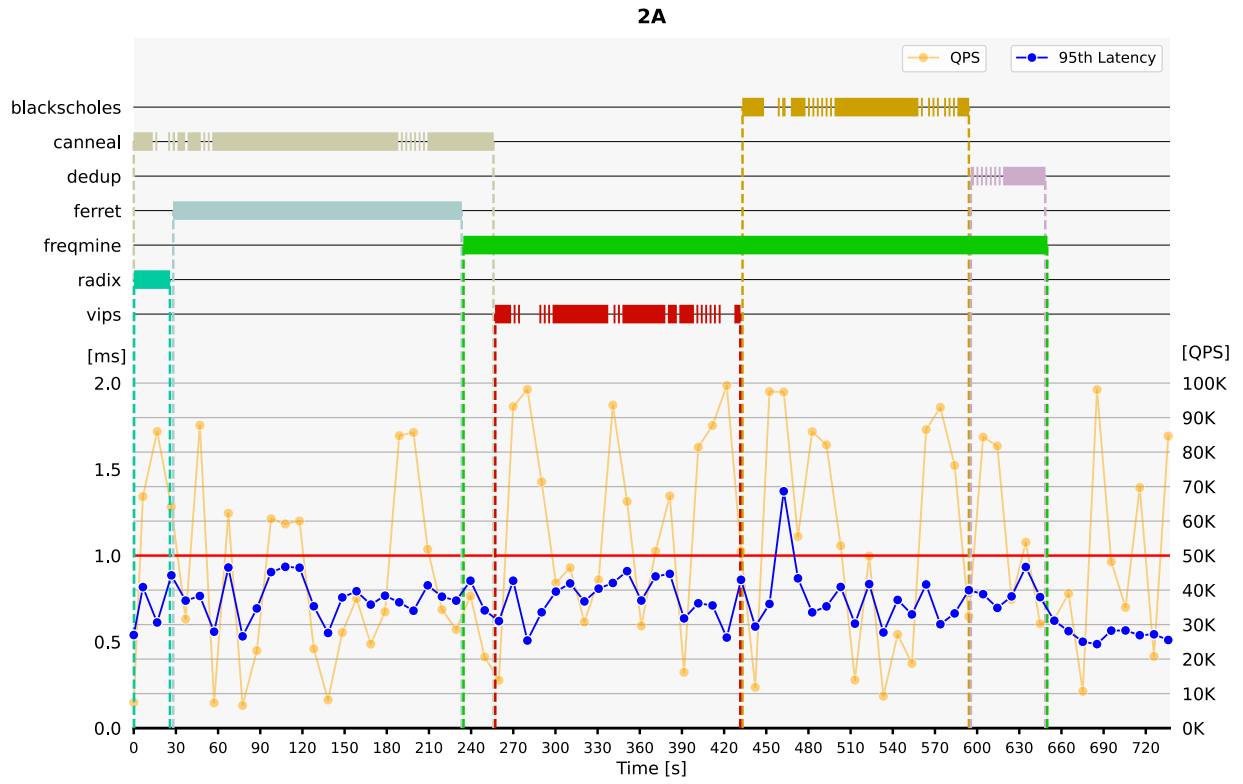
SLO violation ratio for memcached:

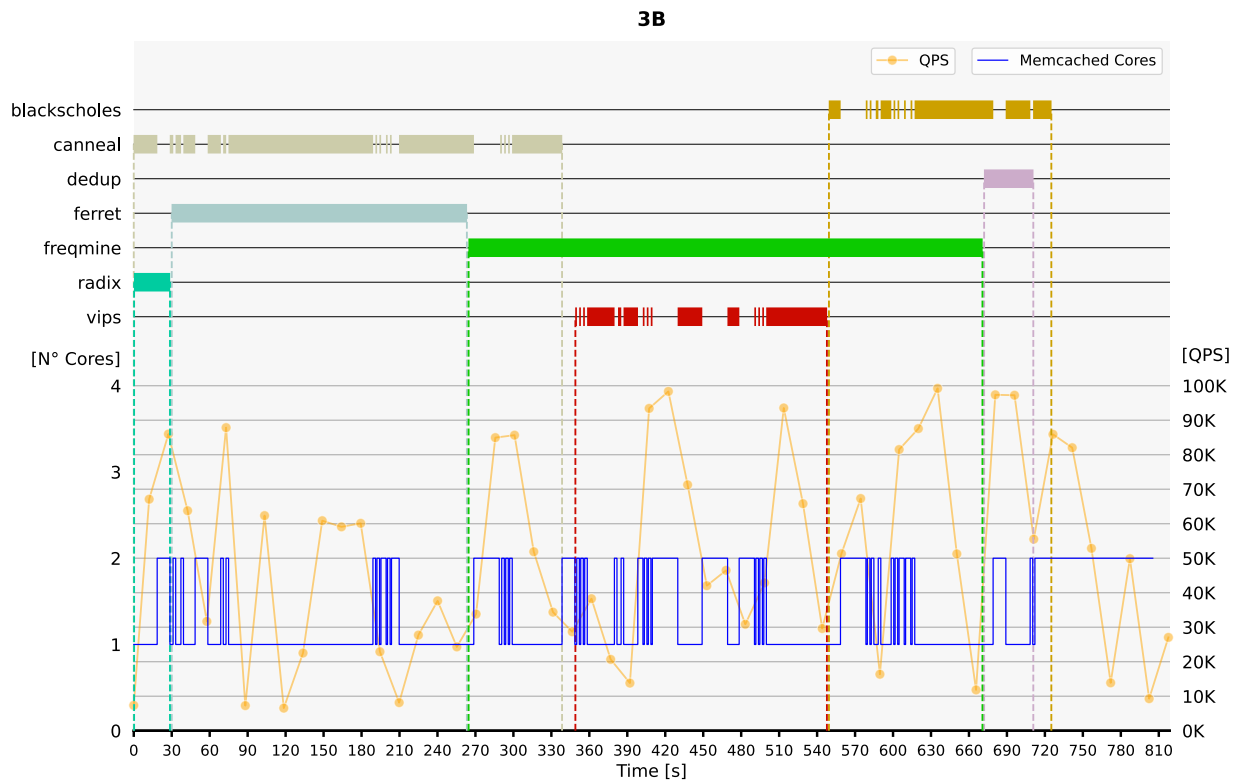
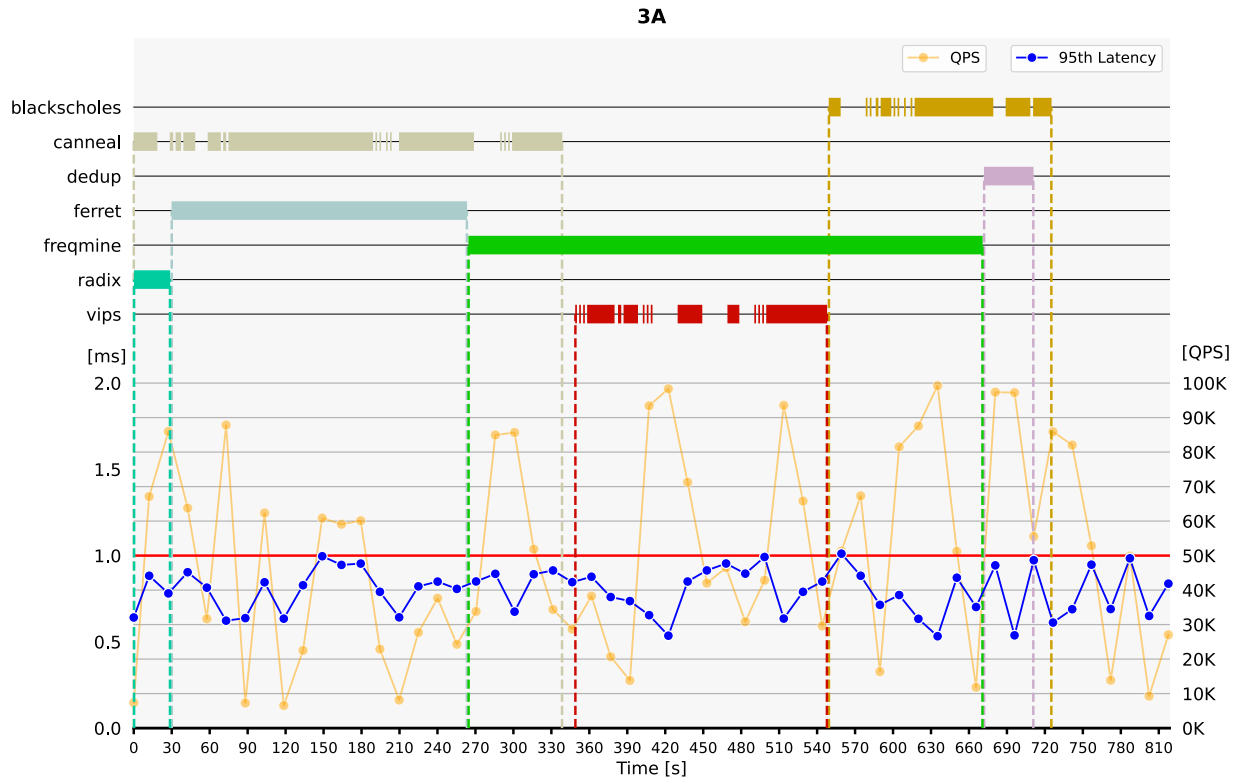
- Run 0: 0% SLO violation
- Run 1: 1.32% SLO violation (1 data point over 76)
- Run 2: 1.32% SLO violation (1 data point over 76)

Include six plots – two plots for each of the three runs – with the following information. Label the plots as 1A, 1B, 2A, 2B, 3A, and 3B where the number indicates the run and the letter indicates the type of plot (A or B), which we describe below. In all plots, time will be on the x-axis and you should annotate the x-axis to indicate which PARSEC benchmark starts executing at which time. If you pause/unpause any workloads as part of your policy, you should also indicate the timestamps at which jobs are paused and unpaused. All the plots will have two y-axes. The right y-axis will be QPS. For Plots A, the left y-axis will be the 95th percentile latency. For Plots B, the left y-axis will be the number of CPU cores that your controller allocates to memcached. For the plot, use the colors proposed in this template (you can find them in `main.tex`).

Plots:







4. [16 points] Repeat Part 4 Question 3 with a modified `mcperf` dynamic load trace with a 5 second time interval (`qps_interval`) instead of 10 second time interval. Use the following command:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
    --noload -T 16 -C 4 -D 4 -Q 1000 -c 4 -t 1800 \
    --qps_interval 5 --qps_min 5000 --qps_max 100000 \
    --qps_seed 3274
```

You do not need to include the plots or table from Question 3 for the 5-second interval. Instead, summarize in 2-3 sentences how your policy performs with the smaller time interval (i.e., higher load variability) compared to the original load trace in Question 3.

Summary: Managing the 5-second interval trace was more challenging than the 10-second one, reflected by higher Service Level Objective (SLO) violation ratios. The first and second experiments recorded SLO violation ratios around 3.5% (5/134), whereas before we had rarely any. The total runtime didn't actually change compared with the 10-second interval trace.

What is the SLO violation ratio for memcached (i.e., the number of datapoints with 95th percentile latency > 1ms, as a fraction of the total number of datapoints) with the 5-second time interval trace?

Answer: The SLO violation ratio for memcached with the 5-second time interval trace is approximately 3.73% (5/134) for the first experiment and 2.98% (4/134) with the 5-second time interval trace. Therefore, we were not consistently under 3%.

What is the smallest `qps_interval` you can use in the load trace that allows your controller to respond fast enough to keep the memcached SLO violation ratio under 3%?

Answer: The smallest interval was 6. We did three different runs, all with SLOs under 3% with the following `qps_interval` values: 2/134 = 1.49%, 4/134 = 2.98%, 2/134 = 1.49%.

What is the reasoning behind this specific value? Explain which features of your controller affect the smallest `qps_interval` interval you proposed.

Answer: The features of the controller that may not be able to respond to faster QPS include the CPU utilization monitoring, the container management, and the dynamic container adjustment strategy as well the taskset command of memcached.

If the QPS changes too frequently, the controller may not be able to keep up with the changes and respond quickly enough to prevent SLO violations. For example, if there is a sudden drop in CPU utilization between two peaks, we may respond to the drop and downscale memcached. However, if the next peak comes, we may not be able to handle the peak and upscale again memcached without violating the SLO. However, if we are not very responsive the time to execute PARSEC jobs will increase, so there should be a balance between responsiveness and stability.

Use this `qps_interval` in the command above and collect results for three runs. Include the same types of plots (1A, 1B, 2A, 2B, 3A, 3B) and table as in Question 3.

Plots:

job name	mean time [s]	std [s]
blackscholes	111.75	2.40
canneal	225.92	0.45
dedup	38.18	2.86
ferret	202.06	0.57
freqmine	411.90	4.81
radix	25.16	0.36
vips	116.26	1.22
total time	665.29	11.33

